

Infix, Prefix, and Postfix Expressions:

For simplicity, we will consider algebraic expressions with binary operators +, −, *, and / only.

Infix Notation:

Usual notation in constructing algebraic expression such that operator appears between two operands; it is ambiguous and requires knowledge of operator hierarchy for its evaluation. Parentheses can also be used to override operator hierarchy.

Prefix Notation:

A special notation in constructing algebraic expression such that operator appears before its two operands. It is unambiguous and does not require the use of parentheses or any knowledge of operator hierarchy for its evaluation.

Postfix Notation:

A special notation in constructing algebraic expression such that operator appears after its two operands. It is unambiguous and does not require the use of parentheses or any knowledge of operator hierarchy for its evaluation.

Example: The infix expression $((a-b)/c)*((d+e)-f)$ has the following postfix and prefix expression.

Postfix: $a\ b\ -\ /\ d\ e\ +\ f\ -\ *$

Prefix: $*\ /\ -\ a\ b\ c\ -\ +\ d\ e\ f$

GIVEN Expression	Prefix Notation	Post fix Notation
$a+b$		
$(a+b)/c$		
$(a+b/c)*d-(e*f/g)$		
$((a/b-c)-d)*(e+f/g)$		
$((a-b)/c)*((d+e)-f)$		

Infix to Postfix Conversion:

```
given a legal infix string;
create an initially empty postfix string;
create an initially empty operator stack S;
for each symbol ch in the infix string do
    if ch is an operand then
        append it to the output postfix string;
    else if ch == '(' then
        push ch onto stack S;
    else if S == ')' then
        pop and append operators to output string until the matching '(' is encountered;
        // discard the two parentheses
    else // ch must be some other operator
        { while operator stack not empty and precedence(top(S)) ≥ precedence(ch)
          and top(S) != '(' do
            pop operator;
            append it to the postfix string;
          end while;
        push S
    }
end for;
while operator stack is not empty do
    pop operator;
    append it to the postfix string;
endwhile;
```

Convert the given infix notation to Postfix notation

Input: $(a-b)/c*(d+e-f/g)$

Postfix conversion:

<u>ch</u>	<u>action</u>	<u>operator stack</u>	<u>Postfix</u>
(	push	(	
a	output	(	a
-	push	(-	a
b	output	(-	a b
)	pop until (		a b -
/	push	/	a b -
c	output	/	a b - c
*	pop		a b - c /
	push	*	
(	push	* (	a b - c /
d	output	* (	a b - c / d
+	push	* (+	a b - c / d
e	output	* (+	a b - c / d e
-	pop	* (	a b - c / d e +
	push	* (-	a b - c / d e +
f	output	* (-	a b - c / d e + f
/	push	* (- /	a b - c / d e + f
g	output	* (- /	a b - c / d e + f g
)	pop until (	*	a b - c / d e + f g /
	pop until empty stack		a b - c / d e + f g / - *

Postfix: $a b - c / d e + f g / - *$

Infix to Prefix Conversion:

```
reverse a given legal infix string;
create an initially empty reversed prefix string;
create an initially empty operator stack S;
for each symbol ch in the reversed infix string do
    if ch is an operand then
        append it to the output prefix string;
    else if ch == ')' then
        push ch onto stack S;
    else if S == '(' then
        pop and append operators to output string until the matching '(' is encountered;
        // discard the two parentheses
    else // ch must be some other operator
        { while operator stack not empty and precedence(top(S)) > precedence(ch)
          and top(S) != '(' do
            pop operator;
            append it to the reversed prefix string;
          end while;
        push S
        }
end for;
while operator stack is not empty do
    pop operator;
    append it to the reversed prefix string;
endwhile;
reverse the reversed output prefix string;
```

Convert the given infix notation to Prefix Notation :

Input: $) g / f - e + d (* c /) b - a ($

Prefix conversion:

<u>ch</u>	<u>action</u>	<u>operator stack</u>	<u>Reversed Prefix</u>
)	push	)	g
g	output	)	g
/	push	) /	g f
f	output	) /	g f /
-	pop	)	g f /
	push	) -	g f / e
e	output	) -	g f / e
+	push	) - +	g f / e d
d	output	) - +	g f / e d + -
(	pop until)		g f / e d + -
*	push	*	g f / e d + - c
c	output	*	g f / e d + - c
/	push	* /	g f / e d + - c
)	push	* /)	g f / e d + - c b
b	output	* /)	g f / e d + - c b
-	push	* /) -	g f / e d + - c b a
a	output	* /) -	g f / e d + - c b a -
(	pop until)	* /	g f / e d + - c b a - / *
	pop until empty stack		

Prefix: $* / - a b c - + d e / f g$